

# **Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C**

## **CSE33 I: Microprocessor Interfacing and Embedded Systems**

### **Lecture#23: I2C and SPI- Serial Peripheral Interface**

Summer, 2023

Mohammad A. Qayum, Ph.D.  
ECE, NSU, dhaka, Bangladesh

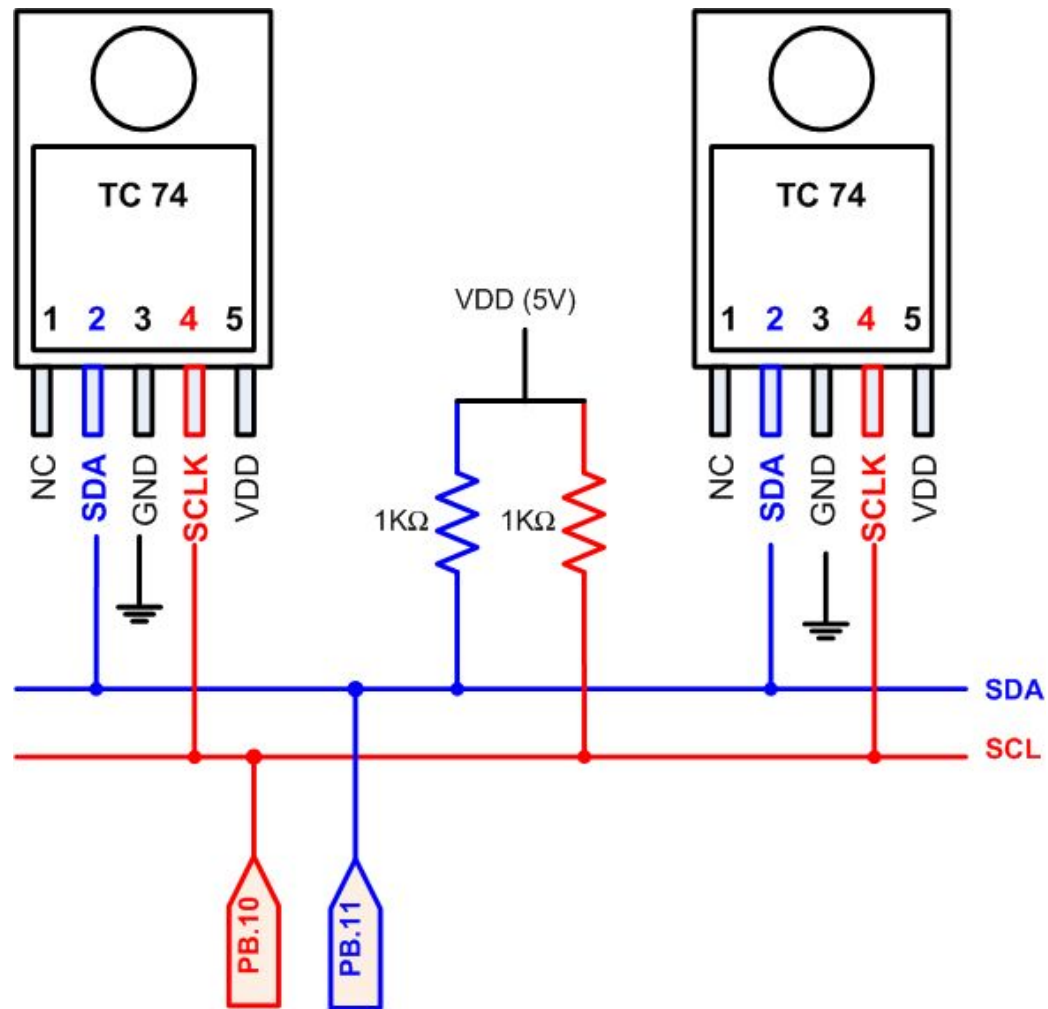
## **EE334 MICROPROCESSOR ENGINEERING II**

### **Lecture#27: Inter-Integrated Circuit (I2C) Chapter 22**

2<sup>nd</sup> November (Friday), Fall 2018

Mohammad A. Qayum, Ph.D.  
ECET, MSNU, Mankato

# Interfacing Serial Digital Thermal Sensor



# I2C Initialization

---

```
void I2C_Initialization(void){
    // Enable the clock of I2C
    RCC->APB1ENR  |= RCC_APB1ENR_I2C2EN;          // I2C 2 clock enable

    // Reset the I2C (set and reset by software)
    RCC->APB1RSTR |= RCC_APB1RSTR_I2C2RST;
    RCC->APB1RSTR &= ~RCC_APB1RSTR_I2C2RST;

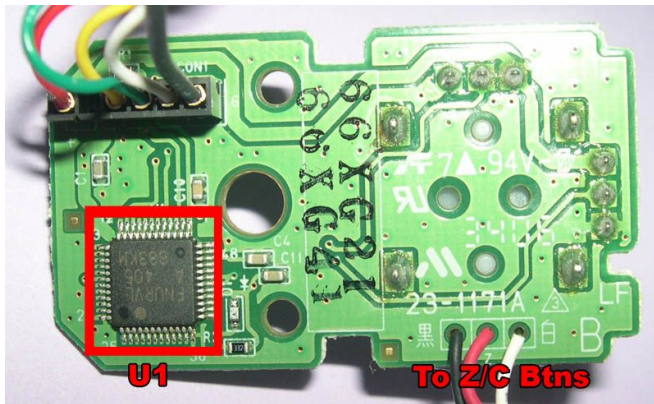
    // I2C Control register. ACK is set and cleared by software, and cleared by hardware when PE=0.
    I2C2->CR1 |= I2C_CR1_ACK;          // Acknowledge after a byte is received
    I2C2->CR1 &= ~I2C_CR1_SMBUS;      // SMBus Mode: 0 = I2C mode; 1 = SMBus mode
    I2C2->CR2 &= ~I2C_CR2_FREQ;       // Clear Peripheral CLK frequency FREQ[5:0]
    I2C2->CR2 |= I2C_CR2_FREQ_1;      // Set Peripheral clock frequency as 2 MHz
    I2C2->CR2 |= I2C_CR2_ITEV TEN;     // Event Interrupt Enable
    I2C2->CR1 &= ~I2C_CR1_PE;         // Disable I2C to configure TRISE
    I2C2->TRISE &= ~I2C_TRISE_TRISE;  // Clear Maximum Rise Time
    I2C2->TRISE |= 17;                // Set Maximum Rise Time for standard mode

    // I2C own address registers
    // Before the STM32 sends its start sequence it sits listening to the I2C lines waiting for its address
    // This is helpful if STM32 is used as slave
    I2C2->OAR1 %= ~(0x01);

    I2C2->OAR1 = 0x00;    // Set up the address byte to select the slave device
    I2C2->OAR2 = 0x00;    // Set up the I2C own address2

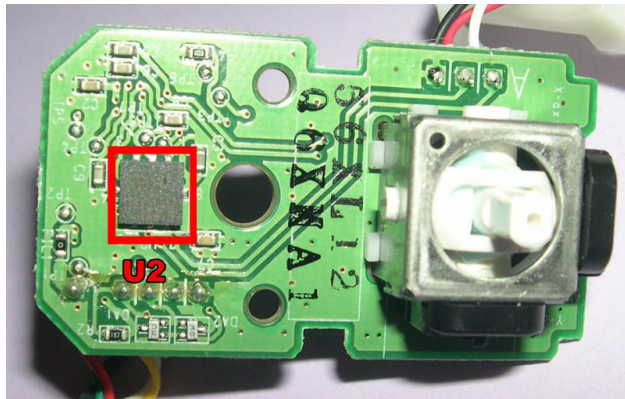
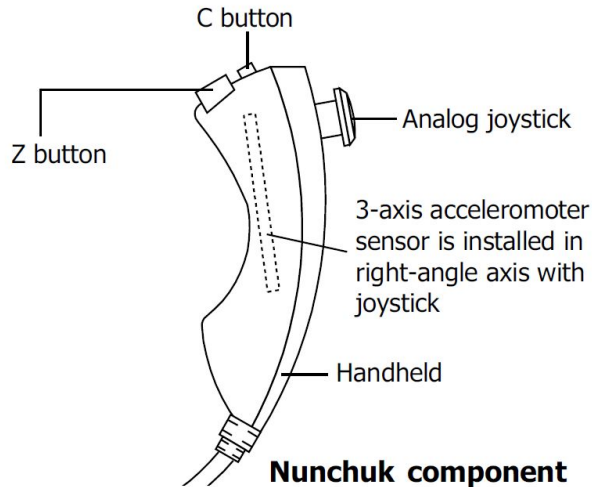
    // I2C Clock control register
    I2C2->CCR &= ~I2C_CCR_FS;    // 0 = Standard Mode ((up to 100 kHz));
                                // 1 = Fast Mode (up to 400 kHz)
    I2C2->CCR &= ~I2C_CCR_CCR;
    I2C2->CCR |= 0x08;
    I2C2->CR1 |= I2C_CR1_PE;    // Enable I2C1
}
```

# Wii Nunchuck



- 3 Axis accelerometer.
- Can capture X,Y and Z motion
- STMicroelectronics **LIS3L02AL**
  - 3.0 V to 3.6 V supply voltage
  - ~2 mW power consumption
  - $\pm 2$  g full-scale
  - 3 acceleration channels plus multiplexed analog output
  - Ratiometric output voltage
  - Power-down mode
  - Embedded self-test
  - 10000 g high shock survivability

# Wii Nunchuck



| Offset | Data                             |                     |                     |                     |                     |                     |             |             |
|--------|----------------------------------|---------------------|---------------------|---------------------|---------------------|---------------------|-------------|-------------|
| 0x00   | Joystick X                       |                     |                     |                     |                     |                     |             |             |
| 0x01   | Joystick Y                       |                     |                     |                     |                     |                     |             |             |
| 0x02   | Accelerometer X (bit 9 to bit 2) |                     |                     |                     |                     |                     |             |             |
| 0x03   | Accelerometer Y (bit 9 to bit 2) |                     |                     |                     |                     |                     |             |             |
| 0x04   | Accelerometer Z (bit 9 to bit 2) |                     |                     |                     |                     |                     |             |             |
| 0x05   | Accel. Z<br>(bit 1)              | Accel. Z<br>(bit 0) | Accel. Y<br>(bit 1) | Accel. Y<br>(bit 0) | Accel. X<br>(bit 1) | Accel. X<br>(bit 0) | C<br>button | Z<br>button |

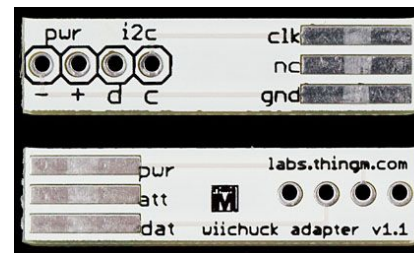
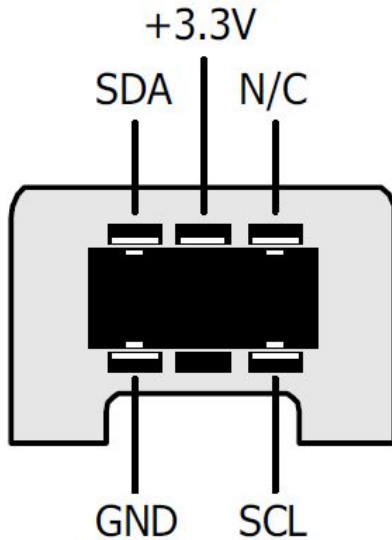
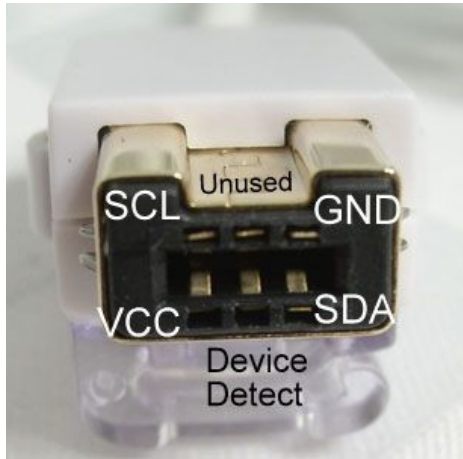
- Two push buttons (labeled as C and Z)
- An 8-bit 2-axis analog joystick (X,Y) and
- A 10-bit 3-axis accelerometer sensor (X,Y, and Z)
- Communication is encrypted.
- One possible decoding is

$$\text{Data} = (\text{Received data XOR } 0x17) + 0x17$$

|              |                                                    |
|--------------|----------------------------------------------------|
| Joystick X   | 0x80 = Center, 0x00 = Full left, 0xFF = Full right |
| Joystick Y   | 0x80 = Center, 0x00 = Full up, 0xFF = Full down    |
| Acceleration | 0 – 1023.                                          |
| Button       | 0 = pressed, 1 = released                          |

□ I2C Standard Mode (100Kbps)

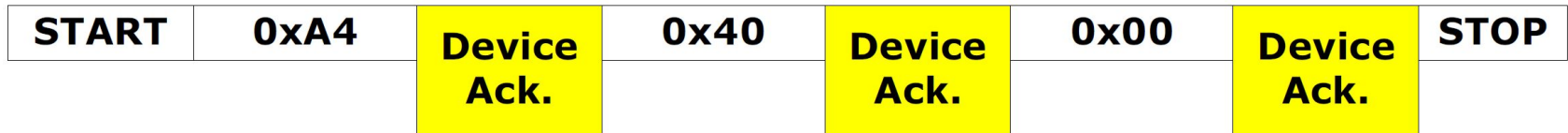
# Interfacing Wii Nunchuck



# Nunchuck Commands

---

## Initialization Command



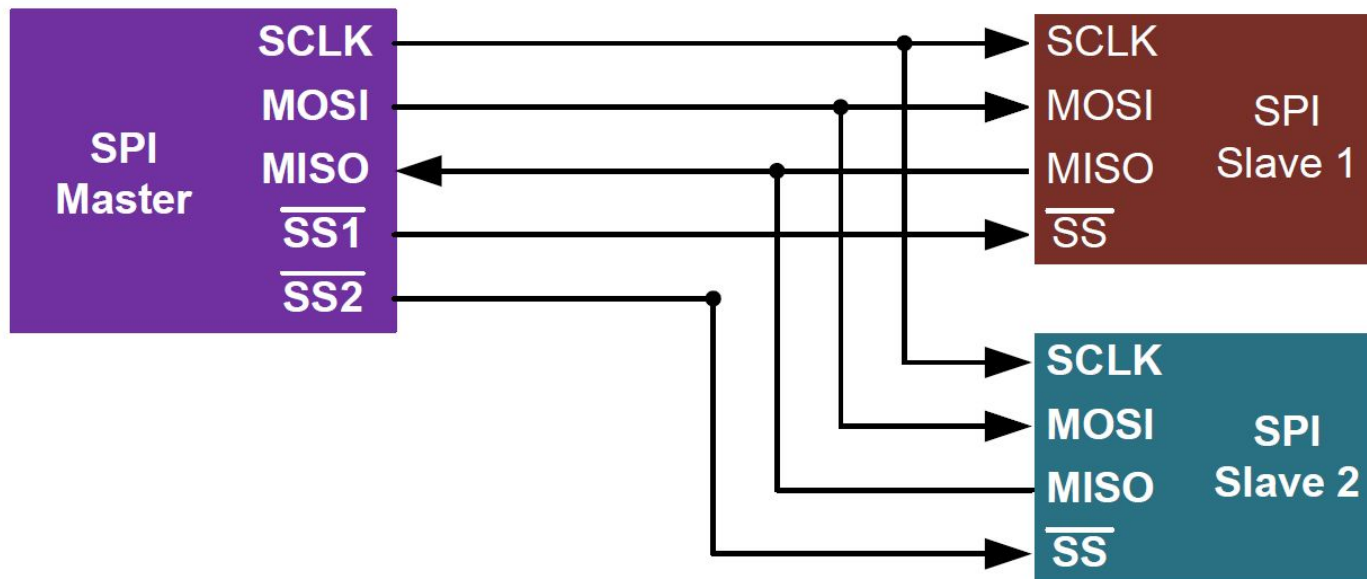
## Conversion Command



- Each Nunchuck has Two Slave Addresses
  - Slave command address: 0xA4
  - Slave data address: 0xA5

# Serial Peripheral Interface (SPI)

- Synchronous full-duplex communication
- Can have multiple slave devices
- No flow control or acknowledgment
- Slave cannot communicate with slave directly.



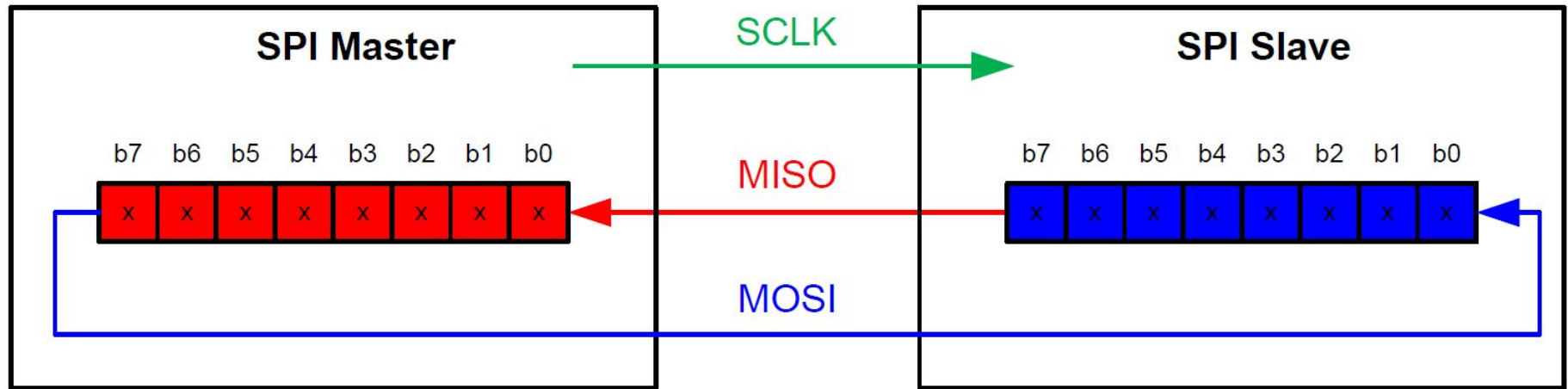
**SCLK**: serial clock

**MOSI**: master out slave in

**SS**: slave select (active low)

**MISO**: master in slave out

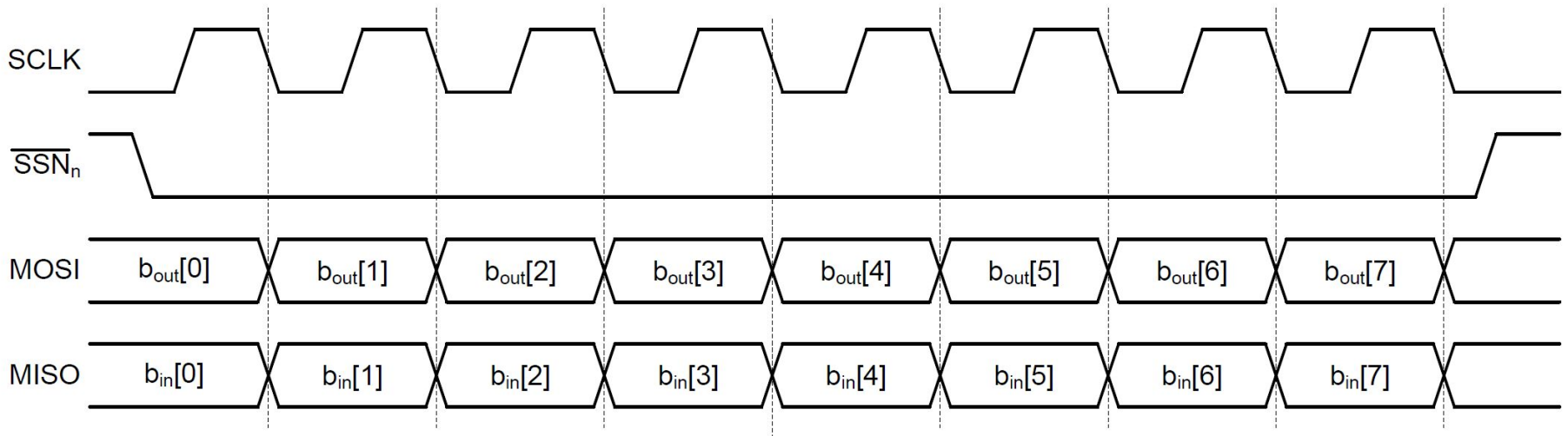
# Data Exchange



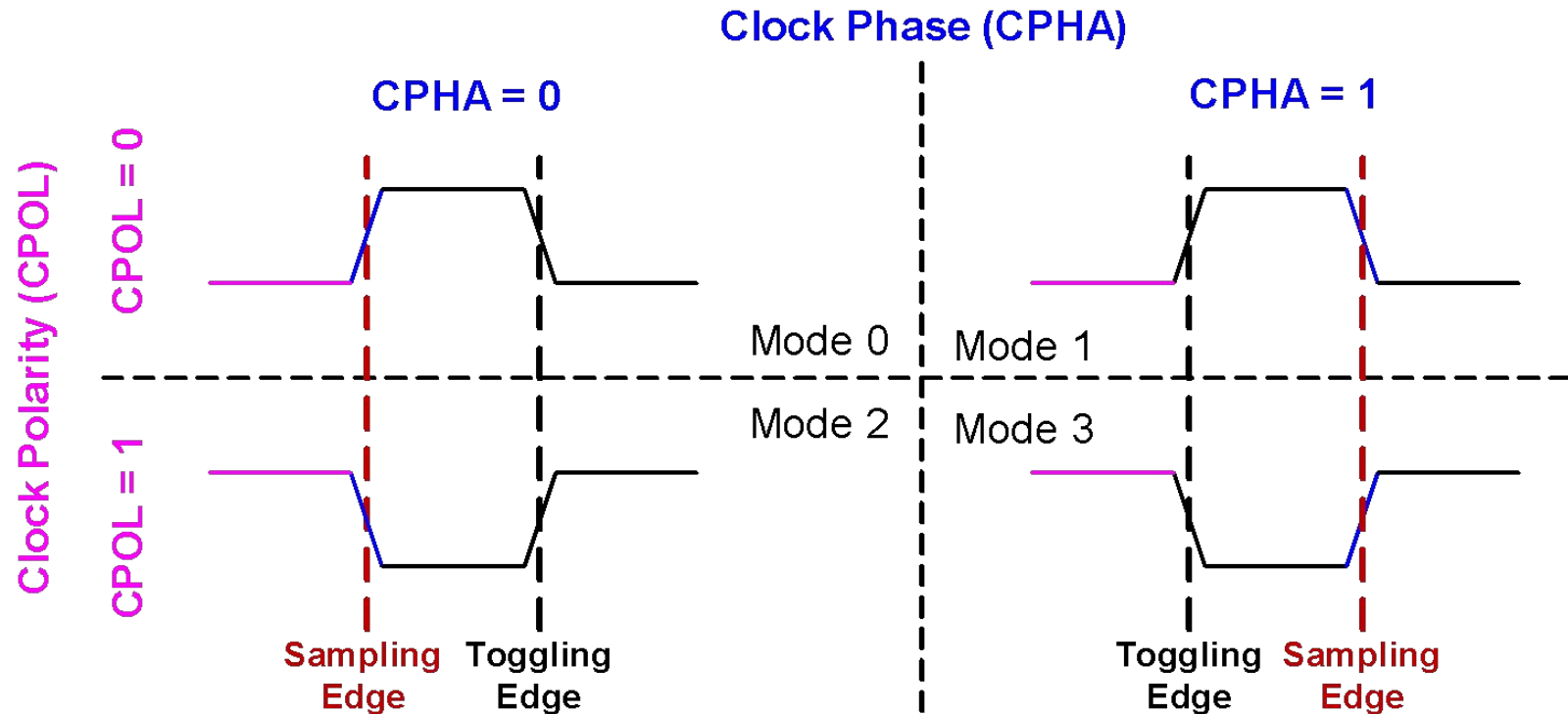
- ❑ Master has to provide clock to slave
- ❑ Synchronous exchange: for each clock pulse, a bit is shifted out and another bit is shifted in at the same time. This process stops when all bits are swapped.
- ❑ Only master can start the data transfer

# Clock

---

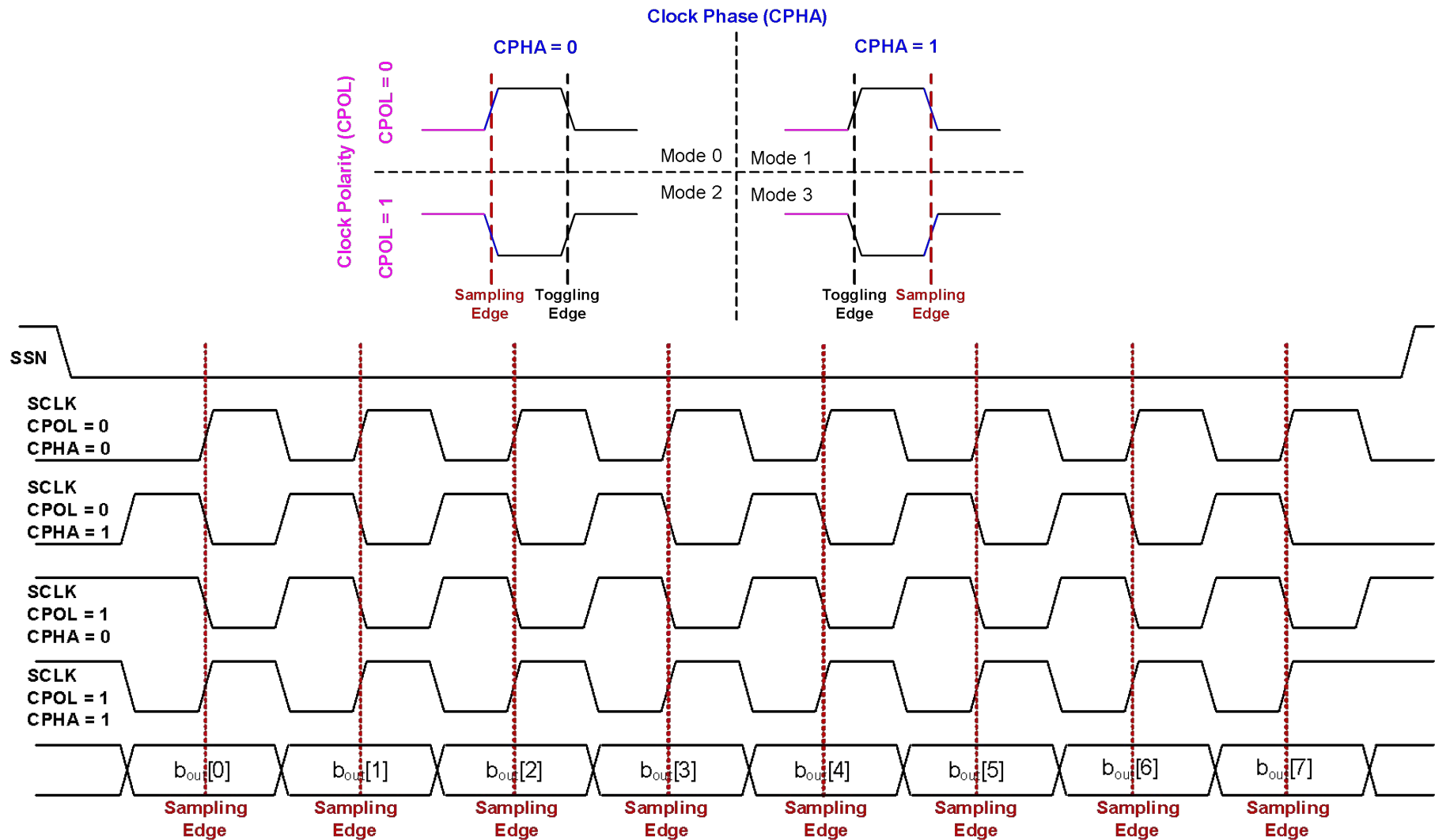


# Clock Phase and Polarity



- CPOL = 0 → SCLK is pushed to low during idle. Otherwise, pulled to high during idle.
- CPHA = 0 → the first clock transition (either rising or falling) is the first data capture edge. Otherwise, the second clock transition is the first data capture edge.
- Combination of CPOL and CPHA determines the clock edge for transmitting and receiving.

# Clock Phase and Polarity



# SPI example: Gyroscope

---

